DATAZOIC

Agentic Platform Implementation Plan

From the current architecture draft to a scalable, permission-aware tool orchestration system

Target workflow

User request → intent analysis → registry selection → hybrid tool retrieval → LLM planning and tool selection → policy/approval → execution → result evaluation → final response

Prepared: 01 November 2026

Scope: local-first implementation using Streamlit, FastAPI, LangGraph, Gemini, Qdrant and SQLite/PostgreSQL

Important scope note This plan is based on the architecture described in the previous discussion and the uploaded two-page programming-task brief. No application source repository was supplied for a code audit, so “already present” means “already described in the architecture”, not verified as implemented in code.

Executive summary

The current architecture has the right foundation: Streamlit for the local UI, FastAPI for the backend, LangGraph for orchestration, Gemini for reasoning, Qdrant for semantic retrieval, SQL storage for operational records, and adapters for HTTP APIs, MCP servers and internal tools. The main change is to make tool discovery an explicit, measurable pipeline instead of one broad “planner/tool discovery” responsibility.

The target system must not send the entire catalogue to the LLM. It should first understand the task, narrow the eligible registries, retrieve a small candidate set using metadata plus hybrid search, and only then ask the LLM to choose or sequence tools. The execution engine remains the only component allowed to invoke tools.

Recommended design Keep LangGraph, Gemini, FastAPI, Streamlit and Qdrant. Refactor the graph into explicit Intent Analyzer, Registry Selector, Tool Retriever, LLM Planner, Policy/Approval, Execution and Result Evaluator nodes. Add a provider-neutral general-tool catalogue, starting with email, calendar, files, web retrieval, calculator/time, database read-only, and system/task search.

1. Requirements and success criteria

The uploaded assessment brief defines the core engineering problem: handle 100+ and potentially thousands of tools without degrading LLM tool-selection quality; demonstrate a PayPal collection with 50+ operations; expose RAG Pipeline and System Search as separate tools; and explain agent structure, routing, state, scale, error handling, framework choices and observability. [Assessment brief, pp. 1–2]

Requirement from brief	Implementation response	Acceptance evidence
100+ to 1,000+ tools	Metadata-first registry routing, hybrid retrieval, top-k candidate set, candidate reranking and evaluation dataset.	Measured Recall@K / NDCG@K and tool-selection accuracy on a fixed test set.
PayPal collection with 50+ APIs	Import/normalize Postman or OpenAPI operations into the same canonical registry; retain provider namespaces.	Representative invoice, transaction, dispute and reporting tasks pass against sandbox.
RAG Pipeline Tool	A separate internal tool for document retrieval and cited answers.	Relevant passages and source citations returned for known questions.
System Search Tool	A separate internal tool for searching tool catalogue, task runs and execution logs.	Can answer “what tools exist?” and “what happened to my last task?” with RBAC filtering.
State and HITL	Persistent LangGraph checkpoint per conversation/task; approval interrupts and resume.	Pause, approve/reject, resume and restart recovery tests pass.
Scalability and error handling	Adapter isolation, bounded concurrency, timeouts, retries for safe operations, idempotency and observability.	Load, timeout, retry, duplicate-action and failure-recovery tests pass.

2. Current architecture baseline: keep, clarify and change

The table below describes the architecture already discussed. It is not a source-code implementation audit.

Area	Already described in the current design	Required change / clarification
Frontend	Streamlit login, chat, approval cards, tools/sources pages, usage dashboard, task logs and admin views.	Keep. Add a task timeline showing intent, registries searched, candidate tools, approval state and execution outcome. Never expose secrets or internal chain-of-thought.
Backend/security	FastAPI gateway, JWT authentication, password hashing, request routing and RBAC.	Keep. Enforce per-user provider connections and tool-level permissions on every execution, not only in the UI.
Orchestration	LangGraph Planner, Tool Discovery, Gemini reasoning, Approval Check, Execution and Recovery.	Refactor into explicit, testable nodes: Intent Analyzer → Registry Selector → Tool Retriever → Candidate Gate → LLM Planner/Selector → Policy Gate → Execution → Result Evaluator loop.
Tool discovery	Qdrant tool catalogue and metadata retriever.	Define canonical metadata, namespace/category filters, exact/keyword + dense retrieval, rank fusion, reranking, thresholds and top-k policy.
Execution	HTTP/API, MCP and internal RAG/System Search execution paths.	Keep a single controlled execution router with provider adapters, JSON Schema validation, timeouts, idempotency and consistent result envelopes.
RAG	Document ingestion, chunking, embeddings, Qdrant retrieval and citation output.	Separate knowledge collections/metadata from tool-catalogue vectors; verify embedding dimension from the configured embedding model instead of hard-coding it.
Storage/telemetry	SQLite or PostgreSQL, execution log, token usage and approval records.	Add conversation/task-run records, provider connection records, tool versions, graph checkpoints and a retrieval evaluation dataset.

3. Target end-to-end workflow

```
Streamlit Chat
  │ authenticated user + conversation/thread ID
  ▼
FastAPI Gateway ── authentication / RBAC / request limits
  ▼
LangGraph START
  ▼
1. Input Normalizer
  ▼
2. Intent Analyzer
  ├─ intent, entities, requested outcome
  ├─ required capabilities, domain hints
  └─ risk class + missing information
  ▼
3. Registry Selector
  ├─ choose one OR multiple namespaces
  └─ filter by tenant, user permission, enabled status
  ▼
4. Tool Retriever
  ├─ exact and keyword search
  ├─ dense semantic search
  ├─ metadata filters + rank fusion
  └─ optional reranker → top candidate tools
  ▼
5. Candidate Gate
  ├─ sufficient match? ── no → broaden once / ask user
  └─ yes
  ▼
6. Gemini Planner / Tool Selector
  ├─ choose tool(s) + schema-valid arguments
  ├─ ask for missing details, or respond without a tool
  └─ produce next action, not executable arbitrary code
  ▼
7. Policy + Argument Validator
  ├─ authorization, JSON Schema, quotas, risk, approval policy
  └─ human approval interrupt when required
  ▼
8. Execution Router
  ├─ HTTP/API adapter  ├─ MCP adapter  └─ Internal tool adapter
  ▼
9. Result Evaluator
  ├─ complete → Final Answer
  ├─ next step needed → loop to retrieval/planner with new context
  └─ error → bounded recovery / user-visible failure
  ▼
10. Final Response + audit/token usage
```

The LLM receives only eligible retrieved candidates, not the full registry. The registry selector may return multiple domains for a compound task. Retrieval proposes candidates; the LLM selects and plans; the policy layer authorizes; the execution router executes. These responsibilities must not be collapsed into a single LLM prompt.

4. Architecture changes by component

4.1 Intent Analyzer

- Input: original user request, recent conversation context, authenticated user and locale/timezone context.
- Output: intent label, task category, entities/parameters, required capabilities, potential domains, expected output, missing fields, risk hint and confidence.
- Use deterministic classification for obvious patterns (calculator, time, exact system status). Use Gemini for ambiguous or multi-domain requests. Do not make a tool call from this node.
- If essential arguments are missing, return a clarification question before tool selection. Do not guess recipient addresses, invoice amounts, IDs or dates.

```
{
 "intent": "email_send",
 "task_category": "communication",
 "required_capabilities": ["email.compose", "email.send"],
 "entities": {"recipient": "anita@example.com", "subject": "Project update"},
 "missing_fields": ["message_body"],
 "risk_hint": "external_write",
 "requires_clarification": true
}
```

4.2 Registry Selector

- Use a controlled taxonomy and registry metadata: payments, communication, calendar, files, web, databases, developer tools, RAG and system administration.
- Allow multiple registry namespaces for compound tasks. Example: search a customer record (database) and inspect a payment (PayPal).
- Apply tenant/user access, connected-account status, enabled/disabled state and environment (sandbox/local) filters before candidate tools reach the LLM.
- Keep “tool discovery” separate from “knowledge retrieval”; the RAG knowledge base is not the tool registry.

4.3 Tool Retriever and ranker

1. Pre-filter the catalogue by registry, capability, provider, user permissions, enabled state and risk policy.
2. Run exact identifier/name matching and keyword/full-text search for API names, IDs and parameter terms.
3. Run dense semantic search over tool name, description, capability and examples.
4. Fuse the ranked result sets. Qdrant supports dense plus BM25/full-text retrieval and RRF/DBSF fusion; start with RRF unless evaluation supports a different choice. [Qdrant hybrid-query documentation]
5. Optionally rerank a limited candidate pool with a cross-encoder or LLM-based scorer, then apply a relevance threshold and return a configurable top-k set.
6. Log candidate IDs, scores, filtering reasons and registry version for every task so retrieval quality can be audited.

Starter retrieval setting Begin with 8–12 candidates after filtering, but tune this using a labelled evaluation set. Use a minimum relevance threshold and allow “no safe match”; do not force the top-k list to contain irrelevant tools.

4.4 LLM Planner / Selector

- Input: user query, normalized intent, relevant conversation context, candidate tools with full JSON input schemas, policy metadata and prior step results.
- Output must be structured: selected tool ID(s), arguments, sequence/dependencies, rationale summary, missing information or a final response. Keep any reasoning trace private; expose a short user-safe action summary.
- Permit the LLM to return “no tool”, “ask user”, or “cannot proceed”. Never fabricate schema fields or silently substitute a different provider/account.
- For multi-step tasks, plan one bounded step at a time or emit a validated plan whose individual steps are re-authorized before execution.

4.5 Execution router and result loop

- Resolve the selected tool ID to the canonical registry record; never execute by free-form model-provided function name alone.
- Validate arguments against the exact registered JSON Schema and provider-specific constraints.
- Dispatch only through an adapter matching source_type: HTTP/API, MCP, or internal.
- Return a canonical envelope: status, output, references, provider request ID, retryability, error code, duration and redacted debug details.
- Return results to LangGraph. Continue only within an iteration/tool-call budget; use idempotency keys for mutating requests and avoid blind retry when the outcome is uncertain.

5. Canonical tool registry design

The registry is the source of truth. Each tool must have both human-readable discovery text and machine-enforceable execution metadata.

Field	Purpose / example
tool_id	Stable ID, e.g. paypal.search_transactions; not just a display name.
name, description	Concise action and when to use it; avoid vague “manage data” descriptions.
namespace / category / capabilities	paypal / payments / transactions.search.
source_type / provider	http_api, mcp or internal; provider/server identifier.
input_schema / output_schema	Versioned JSON Schema; required fields, enums, bounds and examples.
permission_scopes	User roles, provider OAuth scopes and tenant constraints.
risk_level / approval_policy	read, low_write, external_write, financial, destructive; policy key.
enabled / environment	Enabled status, sandbox/production, health and connection status.
timeout / retry / idempotency	Execution limits and safe retry semantics.
version / owner / updated_at	Versioning, provenance and change audit.
search_text / tags / examples	Prepared text used for keyword and embedding retrieval.

Recommended storage collections/tables

- SQL: users, roles, user_tool_permissions, tool_providers, tool_definitions, tool_versions, mcp_servers, provider_connections, conversations, task_runs, execution_logs, token_usage, approval_requests and knowledge_sources.
- Qdrant collection: tool_catalogue_vectors, with tool_id, namespace, provider, capabilities, enabled, tenant visibility and version in payload. Use a separate collection or clearly separate namespace for knowledge chunks.
- Graph checkpoint storage: persistent checkpointer for conversation/task threads. Store IDs and object references in graph state rather than large PDFs, attachments or secrets. LangGraph checkpointers support thread-scoped continuity and resumption after interrupts. [LangGraph persistence and interrupts documentation]

6. General-purpose tool catalogue

Add tools in small, testable groups. Keep provider-independent tool contracts in the registry and put Gmail, Outlook, Google Calendar, Microsoft 365, database and MCP specifics behind adapters. Do not add every possible integration in the first release.

Priority	Tool family	Suggested tools	Policy / notes
P0	System Search	search_tools, get_tool_details, search_task_runs, get_task_status, get_execution_trace_summary	Required by assessment; user-scoped logs and catalogue results only.
P0	Web retrieval	web_search, fetch_url, extract_page_content	Keep search separate from fetch. Restrict private-network/local URLs and apply timeouts.
P0	Calculator and time	calculate_expression, current_datetime, convert_timezone, convert_units	Deterministic tools; no LLM needed for the calculation itself.
P0	Files and documents	list_allowed_files, search_files, read_file, extract_pdf_text, parse_csv, document_metadata	Filesystem access limited to configured project/data folders; start read-only.
P0	RAG Pipeline	ingest_document, search_knowledge_base, answer_with_sources	Separate from tool discovery; return citations/source IDs.
P1	Email	search_emails, get_email, create_email_draft, send_email	Start with one provider (Gmail or Outlook). Sending always shows recipient/subject/body and requires explicit confirmation.
P0	PayPal	search_transactions, get_transaction, get_sales_report, list_invoices, create_invoice, send_invoice, list_disputes, get_dispute	Import representative API operations first, then expand the 50+ collection. Payment/refund/dispute writes need strong approval.
P1	Calendar	list_events, get_availability, create_event, update_event, cancel_event	Show exact attendees, date, time zone and conferencing details before approval.
P1	Contacts	search_contacts, get_contact, create_contact, update_contact	Restrict to the connected user address book; writes require confirmation.
P1	Database read-only	list_tables, describe_table, run_readonly_query, explain_query	Read-only DB role, statement timeout, row limit, allowlisted schemas; never expose arbitrary write SQL.
P1	Git / GitHub	search_repositories, get_issue, list_pull_requests, get_pull_request, create_issue	Start read-only; issue creation is an external write and needs confirmation.
P2	Slack / Teams	search_messages, get_thread, draft_message, send_message	Search respects workspace/user permissions; sending requires preview/approval.

Priority	Tool family	Suggested tools	Policy / notes
P2	Task automation	create_task, list_tasks, get_task_status, cancel_task	Use only for platform-managed jobs; record owner, deadline and idempotency key.

6.1 Email implementation recommendation

Implement a provider-neutral EmailAdapter interface, then one concrete provider first. For Google Workspace, use Gmail API OAuth and request only the scopes required; the Gmail API documents a gmail.send scope for sending on the authenticated user's behalf, and identifies scope sensitivity/restrictions. [Gmail API method and scope documentation] For Microsoft 365, Microsoft Graph exposes POST /me/sendMail and lists Mail.Send as the least-privileged permission for the sendMail operation. [Microsoft Graph sendMail documentation]

- Separate create_email_draft from send_email. Drafting can be prepared without sending; sending is an external write and requires an explicit approval card.
- Approval preview must include To/Cc/Bcc, subject, full body, attachments and sending account. Approval expires and is bound to an immutable payload hash so a modified email needs a new approval.
- Use OAuth authorization-code flow for the signed-in user; encrypt refresh/access tokens at rest. Do not store mailbox passwords or user tokens in the general .env file.
- Track provider message ID and request ID. A timeout after send is an unknown outcome: reconcile against Sent Items or provider state before retrying to avoid duplicate emails.

6.2 MCP integrations: what to add first

For local development, evaluate the official MCP reference implementations for Fetch, Filesystem, Git, Memory and Time. The MCP servers repository explicitly warns that these reference servers are educational examples, not production-ready drop-ins; review and harden each server against the application threat model before enabling it. [MCP servers repository]

- Filesystem: pass only explicit allowed directories; do not mount the whole home directory. Start read-only, and make writes/deletes separate tools with approval. The current filesystem server documentation describes allowed-directory restrictions and warns that write operations can overwrite or move files. [Filesystem server documentation]
- Fetch: use for fetching and extracting public web pages; add a distinct search provider if users need search-engine discovery.
- Git: scope to selected repositories and expose a curated subset of read operations first.
- Memory: optional user-approved preferences/facts only; distinguish user memory from indexed business knowledge.
- Do not add a general unrestricted shell/terminal tool in the default tool pack.

7. LangGraph state and node contracts

Use one persistent thread_id per conversation and a task_run_id per user request. Keep state serializable and small. Store large document content and binary attachments outside graph state; keep object IDs and short summaries in the checkpoint.

```
class AgentState(TypedDict):
  request_id: str
  task_run_id: str
  thread_id: str
  user_id: str
  query: str
  conversation_context: list
  intent: dict
  selected_registries: list[str]
  candidate_tool_ids: list[str]
  candidate_summaries: list[dict]
  plan: list[dict]
  current_step: int
  approval_request_id: str | None
  execution_results: list[dict]
  iteration_count: int
  error_state: dict | None
  final_answer: str | None
  token_usage: dict
```

Recommended nodes: normalize_input, analyze_intent, select_registries, retrieve_tools, validate_candidates, plan_and_select, validate_tool_call, approval_interrupt, execute_tool, evaluate_result, recover_or_stop and compose_final_answer.

Use LangGraph interrupts for actions needing human review and resume the same persisted thread after approval. LangGraph documents that the interrupt pauses execution and the checkpoint preserves state until the graph is resumed. [LangGraph interrupts documentation]

8. Security, approvals and execution policy

Risk tier	Examples	Default behaviour
Read-only / low risk	Calculator, time, search public web, search own allowed files, RAG query, read tool catalogue.	Execute if user and resource authorization pass; log the action.
User-scoped read	Read email, calendar, contacts, private repository or database rows.	Require connected account and least-privilege scopes; enforce ownership and tenant filters.
Reversible write	Create draft, create calendar event, create GitHub issue, write a file in sandbox folder.	Preview details; require confirmation depending on policy and user settings.
External / financial write	Send email/message, send invoice, create payment, refund, submit dispute response.	Always require explicit approval with immutable payload preview; stronger admin policy for financial operations.
Destructive / high impact	Delete files, bulk updates, revoke access, execute arbitrary SQL writes, shell commands.	Disabled by default. Require separate admin enablement, strong confirmation and narrowly scoped execution.

- Authentication and authorization must be rechecked inside the execution service; hiding a button in Streamlit is not a security boundary.

- Never let the LLM choose arbitrary URLs, filesystem roots, database connections, shell commands or provider accounts without policy validation.
- Keep model/provider API keys in environment variables or OS secret storage. Keep per-user OAuth tokens encrypted and access-controlled separately.
- Redact access tokens, passwords, payment details and sensitive message content from logs and LLM traces. Apply retention and deletion policies.
- For mutating tools, define idempotency and reconciliation. Do not blindly retry a request when the provider may have completed it but the response was lost.

9. Observability, audit and usage

Create a trace for every request with correlated request_id, task_run_id and thread_id. Capture timings and structured events for intent classification, selected registries, retrieval candidates and scores, model invocation, selected tool, approval, adapter call and final result.

- User dashboard: tasks, status, tools used, timestamps, token usage and safe result summaries.
- Admin dashboard: all task runs, registry health, tool success/failure rates, latency, retrieval quality, token/cost trends and approval decisions. Apply role-based and tenant-based visibility.
- Prefer local structured logs plus OpenTelemetry-compatible traces for a local-first installation. LangSmith can be added as an optional observability integration; its documentation describes tracing and monitoring, but sending traces to an external service must be opt-in and scrubbed for sensitive data. [LangSmith overview]

10. Implementation roadmap and exit gates

Phase	Work to implement	Deliverables	Exit gate
0. Baseline and contracts	Confirm local run path, environment configuration, auth/RBAC, DB models, API response envelope, error taxonomy and coding conventions. Build a small golden task dataset.	Architecture decision record; config example; baseline tests; sequence diagram.	App starts locally; one authenticated chat request is logged end-to-end.
1. Canonical Tool Registry	Implement registry CRUD, namespaces, JSON schemas, versioning, enabled state, permission/risk metadata and admin UI. Seed internal tools and a small PayPal subset.	tool_definitions schema; registry service/API; admin tools catalogue; import format.	Tools can be registered, versioned, enabled/disabled and retrieved by ID.
2. Intent + Registry Selection	Implement structured intent analysis, taxonomy, capability mapping and multi-registry selection. Add deterministic patterns and LLM fallback.	Intent schema; classifier node; registry selector node; test cases.	Known prompts classify correctly; missing values lead to clarification, not fabricated args.
3. Hybrid Tool Retrieval	Build metadata filters, exact/keyword search, dense retrieval, RRF fusion, optional reranking, top-k and no-match threshold. Index tool metadata in Qdrant.	ToolRetriever service; Qdrant collection/indexing job; retrieval metrics.	Golden queries return expected tools in top-k; unauthorized/disabled tools never appear.
4. LLM Planning + Graph loop	Pass query, intent, candidate schemas and prior results to Gemini; return structured tool selection; add bounded multi-step loop and checkpointing.	LangGraph graph; typed node contracts; checkpoint storage; task timeline events.	Single-tool and multi-tool tasks work; graph resumes after interruption/restart.

Phase	Work to implement	Deliverables	Exit gate
5. Execution + Safety	Standardize HTTP/MCP/internal adapters, schema validation, permissions, timeout, retry, idempotency, approval and redacted audit.	Execution router; policy engine; approval API/UI; adapter tests.	No tool bypasses router; approval-required calls cannot execute before approval.
6. Core tools + PayPal	Add System Search and RAG as separate tools; add calculation/time/files/web tools; import representative PayPal operations and then expand the collection.	Built-in tool pack; PayPal API adapter; sandbox integration tests.	Assessment scenarios pass; citations and system audit queries work.
7. Email and productivity tools	Implement one mail provider with OAuth, draft/send split; add calendar and contacts behind a provider adapter.	EmailAdapter; OAuth connection management; approval preview; calendar connector.	Draft works; send requires approval; sent message reconciles by provider ID.
8. Admin UX + observability	Complete user/admin dashboards, usage/token records, registry health, traces, task logs and retention controls.	Streamlit screens; metrics/events; admin filters; exportable audit record.	A user can inspect their tasks; admin can audit all allowed tasks and usage.
9. Hardening and packaging	Threat-model review, concurrency/load testing, backup/restore, local installer/scripts, data migration and documentation.	Test report; local setup guide; sample .env; backup and recovery instructions.	Clean-machine local setup and acceptance suite pass.

11. Testing and evaluation plan

11.1 Golden query suite

Create a version-controlled evaluation set before tuning retrieval. Include direct matches, ambiguous language, compound tasks, paraphrases, exact API identifiers, unavailable tools, unauthorized tools, missing arguments and adversarial prompt-injection attempts.

Task family	Example evaluation prompt	Expected route
PayPal read	“What was total sales volume last month?”	PayPal reporting tools; correct date range and aggregation.
PayPal invoice	“Create an invoice for $50 for this customer.”	Ask for missing recipient/customer details; draft invoice; approval before send.
Dispute	“Is there an open dispute for user_123?”	Dispute search/details tool; user permission and result grounding.
RAG	“Find the product guide section about API authentication.”	RAG Pipeline Tool; source chunks and citations.
System search	“What tools can manage invoices?”	System Search Tool querying catalogue metadata.
Email	“Draft an email to the vendor about the delivery delay.”	Create draft; no send until explicit approval.
Compound	“Check the latest transaction and email me a summary.”	PayPal retrieval followed by email draft/send approval.
No match	“Control an unsupported device.”	No fake tool call; explain limitation or ask clarification.

11.2 Metrics

- Intent: classification accuracy and clarification precision on the labelled set.
- Retrieval: Recall@5/10, Precision@K and NDCG@K for relevant tool candidates.

- Selection: correct tool choice, correct argument extraction, multi-step plan success and no-tool/clarification accuracy.
- Safety: zero unauthorized calls, zero approval bypasses, correct tenant isolation and no accidental duplicate mutating actions in test scenarios.
- Operations: task completion rate, latency by node/adapter, tool failure rate, timeout/retry rate and tokens/cost per completed task.

Suggested release gate Set numeric retrieval/selection targets only after the first labelled baseline is measured. Safety gates are non-negotiable: unauthorized invocation and approval bypass must remain zero in automated policy tests.

12. Revised project structure

```
application_agent/
├── app/
│   ├── main.py
│   ├── config.py
│   ├── api/
│   │   ├── routes/{auth,chat,tools,mcp,approvals,usage,admin}.py
│   │   └── dependencies.py
│   ├── core/{security,permissions,logging,errors}.py
│   ├── db/{models,session,migrations,vector_store}.py
│   ├── graph/
│   │   ├── state.py
│   │   ├── workflow.py
│   │   └── nodes/{normalize,intent,registry_select,retrieve,plan,policy,execute,evaluate,recover,finalize}.py
│   ├── registry/{models,service,importers,versioning}.py
│   ├── retrieval/{tool_retriever,hybrid_search,reranker,evaluation}.py
│   ├── policies/{tool_policy,approval_policy,schema_validator}.py
│   ├── execution/{router,contracts,retry,idempotency}.py
│   ├── adapters/{http_api,mcp,internal}/
│   ├── integrations/{paypal,gmail,outlook,calendar,github}/
│   ├── tools/{system_search,rag,calculator,time,web,filesystem,database}/
│   ├── services/{token_usage,audit,oauth,secrets,task_runs}.py
│   └── schemas/{auth,chat,tool,approval,usage}.py
├── frontend/
│   ├── app.py
│   ├── components/{chat,login,approval_card,task_timeline}.py
│   ├── pages/{tools,sources,usage,task_logs,user_management,connections}.py
│   └── services/backend_client.py
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── security/
│   └── evals/{golden_queries.jsonl,retrieval_metrics.py}
├── sandbox_storage/
├── scripts/{seed_demo_data,import_openapi,import_postman,reindex_tools}.py
├── .env.example
├── requirements.txt
├── start.bat
└── run_application.bat
```

This is a proposed organization, not a demand to create every folder on day one. Build the modules phase by phase and avoid a large refactor before the contracts and tests exist.

13. Local configuration and operational rules

- Keep app-level secrets such as GEMINI_API_KEY and database configuration in environment variables. Provide .env.example with placeholder values only; never commit .env.
- For external integrations, store OAuth client configuration securely and per-user refresh tokens encrypted in the local credential store/database. Do not reuse one administrator mailbox token for every user.
- Use SQLite for the first single-user/local prototype if concurrency is low; use PostgreSQL when concurrent users, stronger transaction handling, persistent checkpoints or multi-process execution are needed. Keep migrations portable.
- Run Qdrant locally for development. Persist its data directory and provide re-index commands. Keep tool catalogue vectors separate from RAG knowledge vectors.
- Use sandbox/test credentials for PayPal until the full approval, idempotency and audit path passes. Treat local-first as “the application runs locally”; connected Gmail, Outlook and PayPal services still involve network calls to those providers.

14. Key design decisions and trade-offs

Decision	Recommendation	Reason / trade-off
LangGraph vs custom loop	Keep LangGraph.	Clear state transitions, checkpointing, approval interrupts and bounded loops. Adds framework concepts, so keep nodes small and typed.
One giant agent vs staged retrieval	Use staged intent → registry → retrieve → LLM.	Reduces the model’s choice set and gives measurable retrieval diagnostics; adds a few orchestration steps.
Vector-only vs hybrid search	Use metadata + exact/keyword + dense with RRF.	Semantic search handles paraphrases; keyword search handles API names and identifiers. Tune with an evaluation set.
All tools in prompt vs top-k	Send only a small eligible candidate set.	Controls context/token growth and reduces irrelevant choices; a bad retriever must be detected rather than hidden.
MCP for everything vs adapters	Use MCP where it fits; retain HTTP/API and internal adapters.	MCP standardizes tool interoperability but does not replace provider-specific OAuth, business rules or API contracts.
Local-first vs cloud services	Keep core app, DB, Qdrant and logs local; make external observability/integrations optional.	Matches local execution requirement, while email/PayPal necessarily call provider endpoints when enabled.

15. Definition of done

- ☐ User can log in and submit a task from Streamlit chat; FastAPI authenticates the request and assigns request/task/thread IDs.
- ☐ Intent analysis returns a structured task classification and identifies missing information without inventing values.
- ☐ Registry selection can select one or multiple namespaces and filters tools using user permissions and enabled state.
- ☐ Hybrid retrieval returns ranked candidate tool IDs with scores and explanations; the LLM never receives the entire catalogue.

- ☐ Gemini can select a candidate, provide schema-valid arguments, request clarification, or choose no tool.
- ☐ All tool calls pass through a single execution router with schema, permission, risk, timeout and idempotency checks.
- ☐ Human approval can pause and resume the same graph thread; rejected/expired approvals do not execute.
- ☐ PayPal sandbox examples, separate RAG Pipeline Tool and separate System Search Tool work end-to-end.
- ☐ Email draft/send works through one OAuth provider; actual send requires explicit confirmation and is auditable.
- ☐ Multi-step tasks loop safely, stop on completion, and recover from bounded retryable failures.
- ☐ User/admin pages show the correct scoped task history, tool usage, token usage, approvals and errors.
- ☐ Golden retrieval and task tests, policy tests, integration tests and local setup/restore instructions pass.

Appendix A. Example canonical tool record

```
{
 "tool_id": "gmail.send_email",
 "name": "Send email",
 "description": "Send an email using the connected user's mailbox after approval.",
 "namespace": "communication.email",
 "capabilities": ["email.send", "external_write"],
 "source_type": "http_api",
 "provider": "gmail",
 "version": "1.0.0",
 "input_schema": {
  "type": "object",
  "required": ["to", "subject", "body"],
  "properties": {
   "to": {"type": "array", "items": {"type": "string", "format": "email"}},
   "subject": {"type": "string", "maxLength": 250},
   "body": {"type": "string", "maxLength": 20000},
  },
  "additionalProperties": false
 },
 "risk_level": "external_write",
 "approval_policy": "always_confirm_payload_hash",
 "permission_scopes": ["gmail.send"],
 "enabled": false,
 "timeout_seconds": 30,
 "idempotency": "provider_reconciliation_required"
}
```

The record above is illustrative. The approval ID is deliberately not an LLM-supplied argument: the policy service creates and binds approval to the payload, then passes a verified approval context to the executor. Actual provider request fields and OAuth scopes must match the provider API and adapter contract.

Appendix B. Source notes and references

[A1] Uploaded “Datazoic Programming Task” brief, 2 pages. Requirements cited in this plan: scalable handling of 100+ / thousands of tools; PayPal 50+ API scenario; distinct RAG Pipeline and System Search tools; agent structure, routing, state, scalability, error handling and observability (pp. 1–2).

[R1] Qdrant, Hybrid and Multi-Stage Queries. Dense + sparse/BM25 search and RRF/DBSF fusion. Open source

[R2] LangChain, LangGraph Persistence. Thread-scoped checkpoints and stores for continuity, interrupts and fault tolerance. Open source

[R3] LangChain, LangGraph Interrupts. Pause graph execution, checkpoint state and resume using the same thread. Open source

[R4] Model Context Protocol, reference servers repository. Reference server catalogue and warning that examples are not production-ready drop-ins. Open source

[R5] Model Context Protocol, Filesystem Server. Allowed-directory controls and file operation behaviour. Open source

[R6] Google, Gmail API users.messages.send. Send operation and OAuth scopes. Open source

[R7] Google, Gmail API scopes. Scope sensitivity and least-privilege guidance. Open source

[R8] Microsoft Graph, user: sendMail. Mail.Send permission and sendMail operation. Open source

[R9] LangSmith overview. Tracing and monitoring capabilities; optional external observability integration. Open source